# Testing

**Testing:** Testing is the process of evaluating a system, product, or component to verify it meets specified requirements and to identify bugs, defects, or missing functionalities. It ensures reliability, quality, and performance before final delivery. It involves checking intended functionality against actual results to ensure the product works as expected.

## Types of Testing:

1. **Unit tests** check pure logic in isolation,
2. **Widget tests** check UI behavior inside Flutter's test environment,
3. **Integration tests** check the whole app running on a real device or emulator,
4. **Performance profiling** helps you find jank, expensive rebuilds, slow layouts, shader issues, memory pressure, and CPU hotspots.

Flutter's own testing overview recommends many unit and widget tests, plus a smaller number of integration tests for the most important user flows, because speed, confidence, and maintenance cost differ across these layers.

---

### Unit testing with the test package

A **unit test** verifies one function, method, or class without rendering widgets or running the full app. In Flutter projects, these are usually used for business logic such as validation, parsing, formatting, repositories, use-cases, state reducers, and utility methods. Flutter's unit-testing guide says unit tests are best for verifying the behavior of a single function, method, or class, and the core framework behind them is Dart's test package.

#### Where it fits

Use unit tests when the code:

- has no UI dependency,
- should run very fast,
- is deterministic,
- can be tested with mocks/fakes for network, database, or platform layers.

This is the cheapest kind of test to maintain and the fastest to run, but it gives less confidence than widget or integration testing because it does not prove the UI and app wiring work together.

#### Why unit tests matter in Flutter apps

They are ideal for testing:

- form validators,
- mappers from API JSON to model,
- local cache logic,
- state transitions in Bloc/Cubit/Notifier/ViewModel,
- sorting/filtering/pagination logic,
- error handling and retry policy.

### Running unit tests

For Dart packages, the command is dart test. For Flutter apps, the Dart docs explicitly note that Flutter code should be run with flutter test.

Typical commands:

```
flutter test
flutter test test/price_calculator_test.dart
```

---

**Packages and files**

In Dart/Flutter, the conventional place is the test/ directory, and test file names typically end with _test.dart. For pure Dart tests, you use package:test/test.dart; for Flutter code, you often still run tests with flutter test, even though the assertion style is from the test package. (Dart)

**Common API from test**

The test package gives you:

- group() to organize related tests,
- test() to define a test,
- setUp() / tearDown() for preparation and cleanup,
- expect() with matchers such as equals, isTrue, throwsA, contains, etc.

**Example: pure Dart business logic**

```
import 'package:test/test.dart';

class PriceCalculator {

  double total(double price, double taxRate) {

    if (price < 0) throw ArgumentError('price must be >= 0');

    if (taxRate < 0) throw ArgumentError('taxRate must be >= 0');

    return price + (price * taxRate);
```

```
  }
}

void main() {
  group('PriceCalculator', () {
    late PriceCalculator calculator;

    setUp(() {
      calculator = PriceCalculator();
    });

    test('returns price plus tax', () {
      expect(calculator.total(100, 0.18), 118);
    });

    test('throws when price is negative', () {
      expect(() => calculator.total(-1, 0.18), throwsArgumentError);
    });

    test('throws when taxRate is negative', () {
      expect(() => calculator.total(100, -0.1), throwsArgumentError);
    });
  });
}
```

---

### Widget testing

A **widget test** checks how a widget renders and reacts to user input inside a simulated Flutter environment. Flutter's widget testing docs describe it as testing widget classes using tools from flutter_test, which ships with the Flutter SDK.

### What a widget test actually does

It mounts a widget tree in a test harness, lets you:

- find widgets,
- tap buttons,

- enter text,
- pump frames,
- verify rendered output.

It is faster than integration testing because it does not run the full app on a real device, but it gives more confidence than a unit test because UI behavior is involved. Flutter's testing overview places widget tests in the middle: higher confidence and higher cost than unit tests, but still quick.

**What widget tests are great for**

- conditional rendering,
- button taps and callbacks,
- form validation messages,
- navigation trigger checks,
- loading/success/error UI states,
- animations and transitions,
- list rendering,
- theme and localization dependent text.

**Core tools from flutter_test**

Common pieces:

- testWidgets() to define widget tests,
- WidgetTester to interact with the widget tree,
- pumpWidget() to render the widget,
- pump() / pumpAndSettle() to advance frames and animations,
- find.text, find.byType, find.byKey to locate widgets,
- expect() to verify outcomes. (Flutter Documentation)

**Example: counter widget**

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

class CounterWidget extends StatefulWidget {
  const CounterWidget({super.key});
```

```
  @override
  State<CounterWidget> createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Column(
          children: [
            Text('$count', key: const Key('counterText')),
            ElevatedButton(
              onPressed: () => setState(() => count++),
              child: const Text('Increment'),
            ),
          ],
        ),
      ),
    );
  }
}

void main() {
  testWidgets('increments counter when button is tapped',
      (WidgetTester tester) async {
    await tester.pumpWidget(const CounterWidget());
```

```
    expect(find.text('0'), findsOneWidget);
    expect(find.text('1'), findsNothing);

    await tester.tap(find.text('Increment'));
    await tester.pump();

    expect(find.text('1'), findsOneWidget);
  });
}
```

---

**Integration testing**

An **integration test** verifies that multiple parts of the app work together while the app runs on a target device or emulator. Flutter's integration testing docs say these tests run on the target device, can be launched from the host with flutter test integration_test, and use flutter_test APIs in a style similar to widget tests.

**Why integration tests exist**

Unit and widget tests do not fully validate:

- real navigation stacks,
- plugin interactions,
- device-specific behavior,
- app lifecycle,
- end-to-end flows like login → fetch data → checkout,
- performance on real hardware.

Integration tests give the **highest confidence**, but they are the slowest and costliest to maintain. Flutter's testing overview says exactly that trade-off.

**How it differs from widget tests**

The syntax looks similar, but the environment is different:

- widget tests run in a simulated Flutter test environment,
- integration tests run the app on a device/emulator,
- plugins, rendering, timing, I/O, and lifecycle are closer to real behavior.

**Typical use cases**

Use integration tests for:

- sign in / sign up flows,
- deep links,
- checkout and payments UI flow,
- onboarding,
- navigation across many screens,
- offline/online transitions,
- plugin-heavy flows such as camera, maps, storage, notifications,
- release-critical smoke tests.

---

**Performance profiling:**

Performance profiling is the dynamic analysis of a program's runtime behavior—using tools to measure CPU usage, memory allocation, and function execution time—to identify performance bottlenecks. It maps hardware metrics to source code, helping developers optimize efficiency, reduce latency, and improve application responsiveness.

**Key Aspects of Performance Profiling**

- **Types of Profilers:**
    - **Sampling Profilers:** Periodically interrupt the CPU to record the active instruction, resulting in low overhead

.

    - **Instrumentation Profilers:** Add code to the application to measure the exact time spent in functions, offering high precision but higher overhead.
- **Performance Metrics:** Key measurements include execution time for routines/functions, frequency of function calls, and hardware performance monitoring (HPM) data.
- **Common Bottlenecks Identified:** High CPU usage, memory leaks, inefficient algorithms, and slow I/O operations.

**Common Issues and Solutions**

- **UI Thread Lag (Bottom Graph):** Occurs when Dart code is too expensive.
    - **Fix:** Move heavy computations to a separate Isolate or optimize build() methods to be lighter.

- **Raster Thread Lag (Top Graph):** Occurs when the scene is too complex to render.
    - **Fix:** Reduce the use of expensive effects like Opacity, Clip, or Shadows. Use RepaintBoundary to cache static, complex parts of the UI.
- **Shader Compilation Jank:** Appearing as dark-red frames, this happens the first time a specific animation or effect is rendered.
    - **Fix:** Enable the Impeller rendering engine (default in recent versions for iOS and Android) to pre-compile shaders.